

# Cours

# Systèmes d'information distribués

Prof. Abderrahmane BAADACHE

`abderrahmane.baadache@gmail.com`

Novembre, 2021

# 1. Introduction sur les systèmes d'information

ooo

---

## a. Définition

- **Un SI (Système d'information)** est un ensemble organisé de ressources (données, procédures, matériel, logiciel, etc.) permettant d'acquérir, de stocker, de structurer et de communiquer des informations sous forme de textes, images, sons, ou de données au sein des organisations.
- On distingue des systèmes d'information :
  - **Supports d'opérations** (traitement de transaction, contrôle de processus industriels, supports d'opérations de bureau et de communication)
  - **Supports de gestion** (aide à la production de rapports, aide à la décision, etc.).
- **Exemple** : Système d'information médical.

- Bases de données (dossiers médicaux + l'administratif), infrastructure réseau.
- Applications médicales de gestion : administrative, médicaments, équipements, examens, planification, etc.

# 1. Introduction sur les systèmes d'information

ooo

---

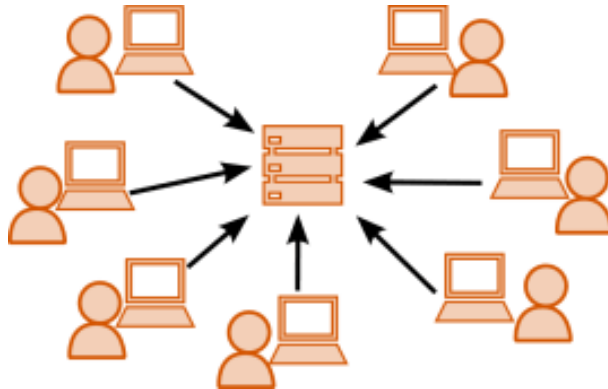
## b. Fonctions d'un SI

- **Échange de données** entre applications hétérogènes manipulant des données au format hétérogènes.
- **Répartition** (processus et données) sur des sites géographiques distants.
- **Interopérabilité** des plates-formes.
- **Portabilité** des applications.
- Gestion de la **réplication** : Serveurs de fichiers répliqués, bases de données répliquées, Site Web miroirs.
- Gestion de la **cohérence des données**.
- Gestion des **accès concurrents**.
- **Persistance** des données.
- **Tolérance aux pannes**.
- **Scalabilité** : performance garantie si nombre de sites, distance entre les sites, nombre d'utilisateurs, nombre de processus, volume des données et fréquence des interactions augmentent.
- **Sécurité** (Authentification, intégrité, confidentialité, anonymat, contrôle d'accès).

# 1. Introduction sur les systèmes d'information

ooo

## c. Systèmes centralisés



- Composants localisés sur un site unique.
  - **1 processeur** : une horloge commune.
  - **1 mémoire centrale** : un espace d'adressage commun.
  - **1 système d'exploitation** : gestion centralisée des ressources et un état global facilement reconstituable.
  - **Accès local aux ressources**
- Centralisation des données et des traitements.
- Presque abandonné car n'est pas pratique.

# 1. Introduction sur les systèmes d'information

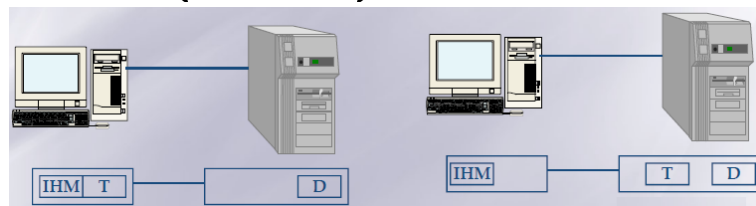
ooo

---

## d. Client-serveur

Client-serveur désigne une interaction à travers un réseau entre plusieurs processus, l'un qualifié de **client**, envoie des requêtes. L'autre, qualifié de **serveur**, attend les requêtes des clients et y répond.

- 2 niveaux (2-tier)



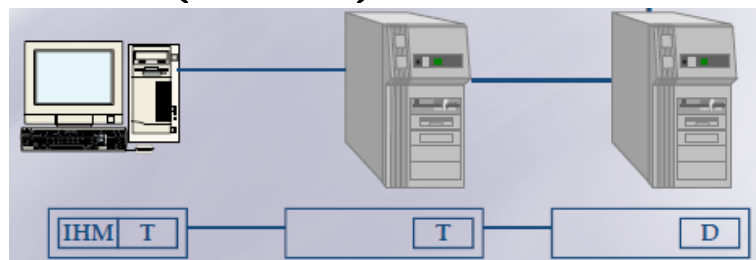
- ⇒ Architecture dénommée **client lourd**
  - Le poste de travail héberge l'ensemble de la gestion d'interface homme-machine et le traitement.
  - Le serveur est un serveur de base de données.
- ⇒ Architecture dénommée **client léger**
  - Le poste de travail n'héberge que l'interface homme-machine.

- Le serveur héberge les données et les traitements.

### Avantages & inconvénients

- + Mise en œuvre relativement simple.
- + Efficace pour un nombre réduit de clients.
  - Coûts de déploiement et de MAJ.
  - Accès concurrent limité.

- **3 niveaux (3-tier)**



- Le poste de travail héberge l'interface homme-machine et une partie des traitements.
- Le serveur d'applications gère l'autre partie des traitements.
- Le serveur de données gère les accès aux données.
- Architecture dénommée `traitements coopératifs`.

## Avantages

- Centralisation des traitements.
- Pas de duplication des données.
- Gestion simple de la cohérence des données.



# 1. Introduction sur les systèmes d'information

ooo

---

## e. Quelques rappels

- Une **adresse IP** est un quadruplet de nombres de 8 bits chaque, séparés par des points. Exemple : 192.168.1.10
- **@IP 127.0.0.1** : adresse de rebouclage (ang. loopback) qui désigne la machine locale (ang. localhost).
- **5 classes**
  - \* **Classe A** : de 1.0.0.1 à 126.255.255.254.  
Le 1er octet réseau (NetID), les 3 octets suivants pour l'@ de l'hôte (hostID))
  - \* **Classe B** : de 128.0.0.1 à 191.255.255.254.  
Les deux 1ers octets réseau, 2 octets suivants pour l'@ de l'hôte.
  - \* **Classe C** : de 192.0.0.1 à 223.255.255.254.  
Les trois 1ers octets réseau, 4ème octet pour l'@ de l'hôte

- \* **Classe D** : de 224.0.0.1 à 239.255.255.255.  
Groupes multicast.
  - \* **Classe E** : réservées pour futur usage.
- **IPv6** résout le problème de pénurie d'adresses IP. @IP sur 128 bits.

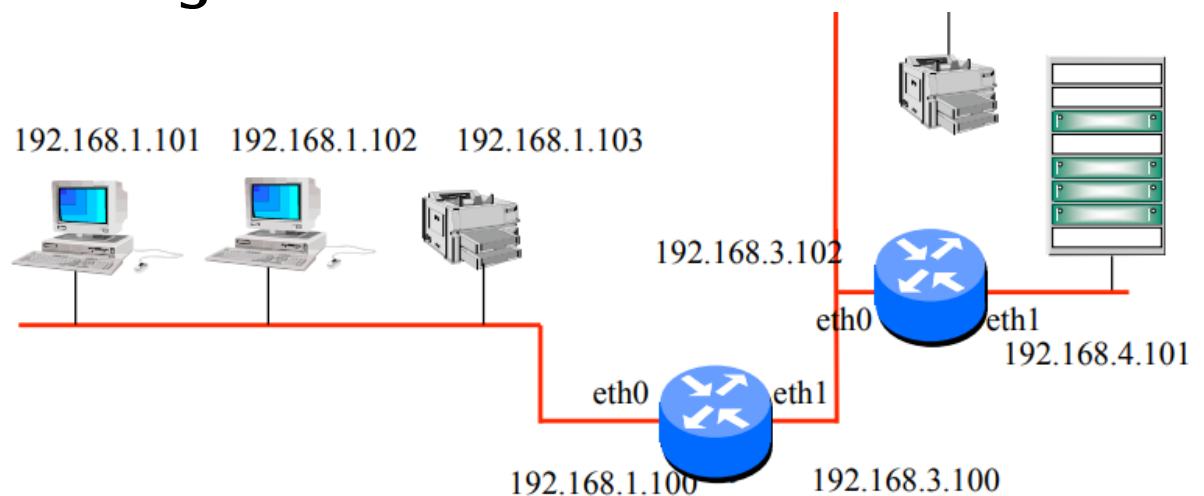
# 1. Introduction sur les systèmes d'information

ooo

## — Port d'écoute

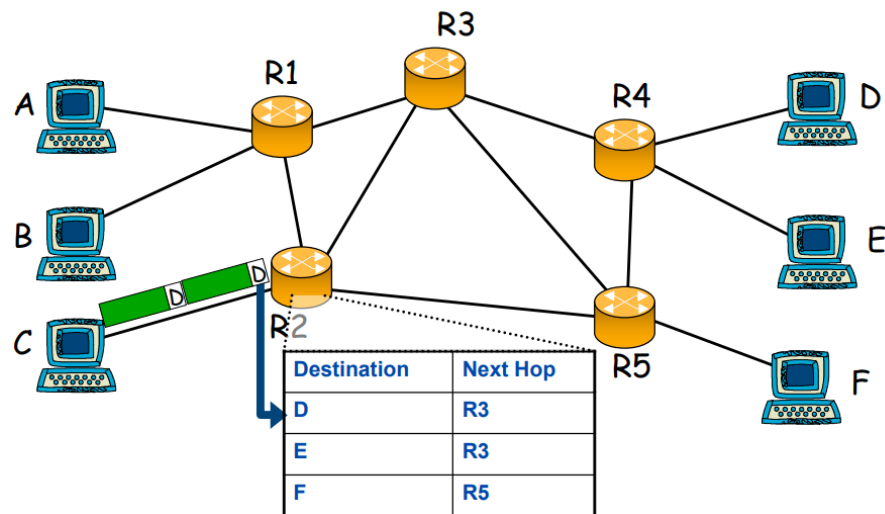
- \* 16-bits unsigned integer, de 1 à 65535.
- \* Ports utilisateur  $\geq 1024$ .
- \* Exemples de ports réservés : FTP : 21/TCP, Telnet : 23/TCP, SMTP : 25/TCP, Login : 513/TCP, HTTP : 80/TCP.

## — Routage dans les réseaux IP



- Le **routage** est l'opération qui consiste à trouver un chemin entre deux machines, l'une émettrice et l'autre réceptrice.

- Un routeur est un matériel réseau spécifique, conçu spécialement pour le routage.
- Les routeurs se réfèrent à des **tables de routage** internes pour prendre des décisions d'acheminement des paquets le long des chemins du réseau.



- Le routage peut être **statique** ou **dynamique**
  - ★ En statique, un administrateur réseau établit manuellement des tables de routage statiques.

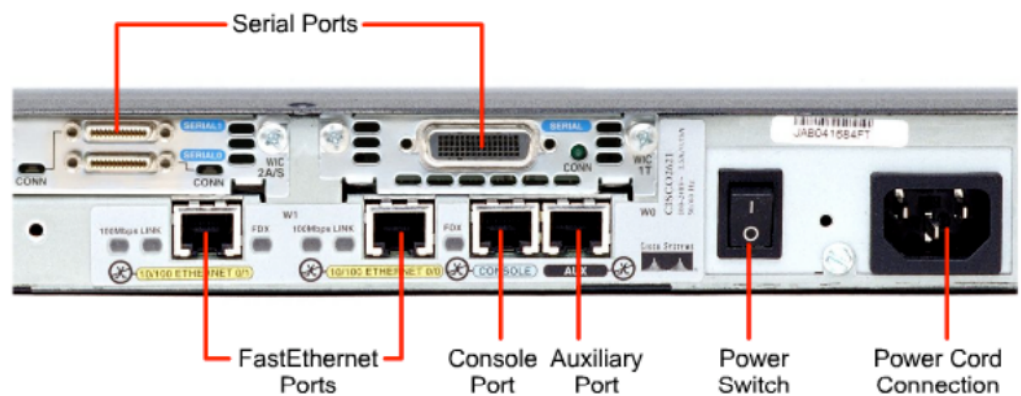
- ★ Les tables de routage dynamiques se mettent à jour automatiquement.

— Quelques protocoles de routage

- ★ **IP** : l'Internet Protocol (IP) spécifie l'origine et la destination de chaque paquet de données. Les routeurs inspectent l'en-tête IP de chaque paquet pour identifier où les envoyer.
- ★ **OSPF** : le protocole OSPF (Open Shortest Path First) est couramment utilisé par les routeurs de réseau pour identifier dynamiquement les routes disponibles les plus rapides et les plus courtes pour envoyer des paquets à leur destination.
- ★ **RIP** : le protocole RIP (Routing Information Protocol) utilise le "

comptage de sauts " pour trouver le chemin le plus court d'un réseau à un autre.

— Connections du **2600 router**



## 2. Protocole HTTP et programmation serveur

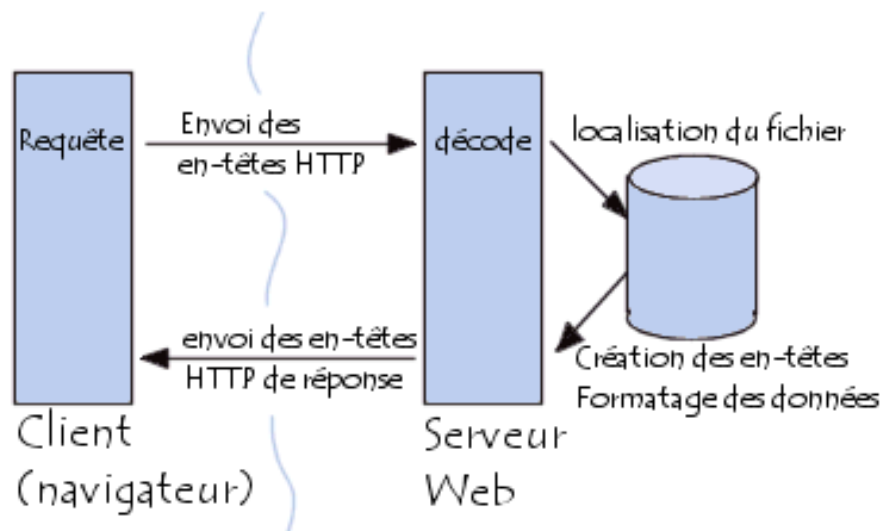
ooo

---

### a. Protocole HTTP

- **HTTP** (Hypertext Transfer Protocol) est un protocole de communication entre un *client* et un *serveur* pour le *World Wide Web*.
- **HTTP** permet un transfert de fichiers localisés grâce à une chaîne de caractères appelée **URL** entre un navigateur (le client) et un serveur Web.
- **Client Web** : un programme qui tourne sur un ordinateur, il envoie des requêtes à un serveur qui lui répond en lui envoyant un document (HTML, XML, etc.)
- **Serveur HTTP** ou **serveur Web** : est un logiciel servant des requêtes respectant le protocole de communication HTTP.
- **Serveurs HTTP les plus utilisés** :
  - Apache HTTP Server de la Apache Software Foundation.

- Internet Information Services de Microsoft.
- Sun Java System Web Server de Sun Microsystems.
- Le serveur Web Zeus de Zeus Technology.
- **Communication entre client et serveur** : se fait en deux temps



- **Requête HTTP** : Elle comprend
  - Une ligne de requête (méthode, URL, version du protocole)
  - Les champs d'en-tête de la requête.
  - Le corps de la requête



```
METHODE URL VERSION<crlf>
EN-TETE : Valeur<crlf>
.
.
.
EN-TETE : Valeur<crlf>
Ligne vide<crlf>
CORPS DE LA REQUETE
```

## — Exemple de requête HTTP

```
GET [/ https://www.commentcamarche.net/] HTTP/1.0
Accept : text/html
If-Modified-Since : Saturday, 15-January-2000 14:37:11 GMT
User-Agent : Mozilla/4.0 (compatible; MSIE 5.0; Windows 95)
```

## — Exemples de commandes et d'entêtes

### — Commandes :

- **GET** : Récupérer une ressource.
- **HEAD** : Récupérer l'entête de la ressource.
- **POST** : Envoi des données.

### — Entêtes :

- **Accept** : Type de contenu accepté.

- **Authorization** : identification du navigateur.
- **Date** : date début de transfert.
- **Réponse HTTP** : Elle comprend
  - **Ligne de statut** : précisant la version du protocole utilisé et l'état du traitement de la requête.
  - **Champs d'en-tête de la réponse** : donne des informations supplémentaires sur la réponse et/ou le serveur.
  - **Corps de la réponse** : contient le document demandé.

```
VERSION-HTTP CODE EXPLICATION<crlf>
EN-TETE : Valeur<crlf>
.
.
.
EN-TETE : Valeur<crlf>
Ligne vide<crlf>
CORPS DE LA REPONSE
```

- **Exemple de réponse HTTP**

```
HTTP/1.0 200 OK
Date : Sat, 15 Jan 2000 14:37:12 GMT Server : Microsoft-IIS/2.0
Content-Type : text/HTML
Content-Length : 1245
Last-Modified : Fri, 14 Jan 2000 08:25:13 GMT
```

- **Exemples d'en-têtes de réponse**
  - **Content-Encoding** : Type de codage.
  - **Content-Language** : Type de langage.
  - **Content-Length** : Longueur du corps de la réponse.
- **Exemples de codes de réponse** : Ce sont les codes renvoyés par le navigateur, lorsqu'il n'arrive pas à fournir la page demandée.
  - **200** : message **OK**, i.e., la requête a été accomplie correctement.
  - **202** : message **ACCEPTED**, i.e., la requête a été acceptée.
  - **204** : message **NO RESPONSE**, i.e., le serveur a reçu la requête mais il n'y a pas d'information à renvoyer.

- **301** : message **MOVED**, i.e., Les données demandées ont été transférées à une nouvelle adresse.
- **400** : message **BAD REQUEST**, i.e., la requête est impossible à satisfaire.
- **404** : message **NOT FOUND**, i.e., requête bien reçu, mais réponse correspondante non trouvée.

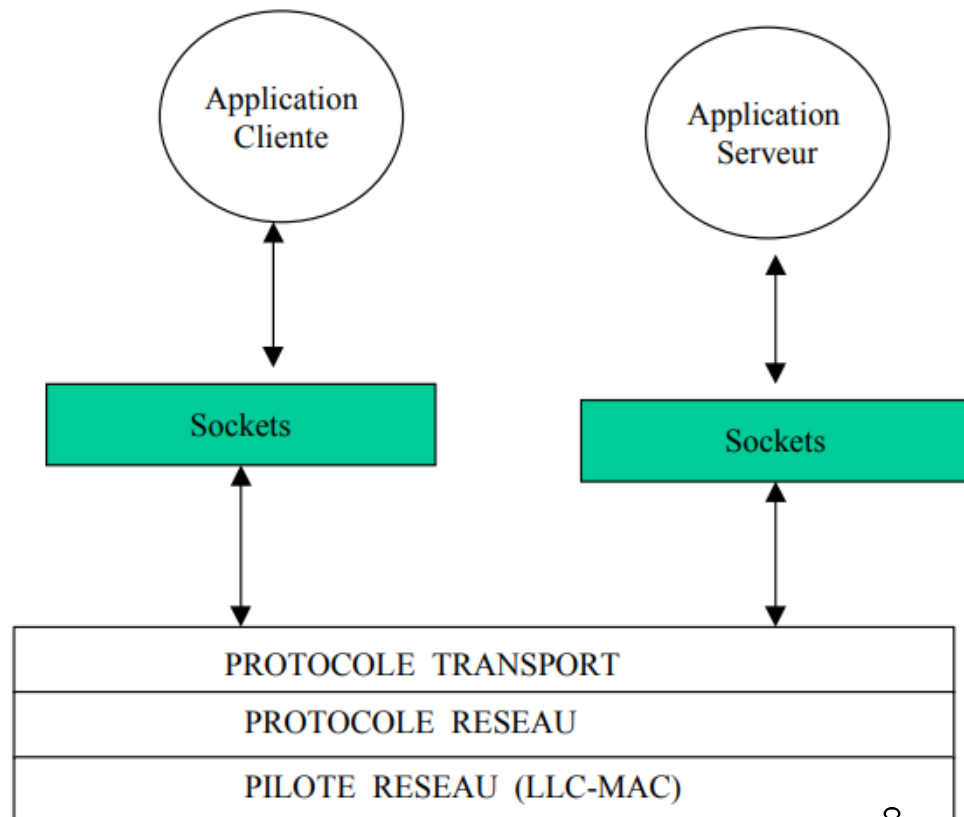
## 2. Les sockets

ooo

---

### a. Notion de sockets

- Il s'agit d'un modèle permettant la communication inter processus (IPC - Inter Process Communication).
- En d'autres termes, une **socket** est un point de communication par lequel un processus peut émettre ou recevoir des données.



## 2. Les sockets

ooo

---

- Pour établir une communication vers une machine distante, il faut :
  - \* s'attribuer un **numéro de port**.
  - \* déterminer l'**adresse Internet de la machine réceptrice**.
  - \* déterminer le **numéro de port du service distant**.
- Modes de communication
  - \* Le **mode connecté**, comparable à une communication téléphonique, utilisant le protocole **TCP**, dans lequel une connexion durable est établie entre les processus.
  - \* Le **mode non connecté**, analogue à une communication par courrier, utilisant le protocole **UDP**. Il s'agit d'une communication sans connexion, rapide mais moins fiable.
- Types de socket

- \* Stream Sockets (TCP) : communication en mode connecté.
- \* Datagram Sockets (UDP) : communication en mode non connecté.

## 2. Les sockets

ooo

---

### b. Fonctionnement des sockets

⇒ **Coté client** (en mode connecté)

1. crée une socket.
2. se connecte au serveur en donnant l'adresse socket distante (adresse Internet du serveur et numéro de port du service).
3. lit ou écrit sur la socket.
4. ferme la socket.

⇒ **Coté serveur** (en mode connecté)

1. crée une socket.
2. associe une adresse socket (son adresse Internet et le numéro de port choisi) au service : binding.
3. se met à l'écoute des connexions entrantes.
4. Pour chaque connexion entrante :
  - accepte la connexion.
  - lit ou écrit sur la nouvelle socket.
  - ferme la nouvelle socket.

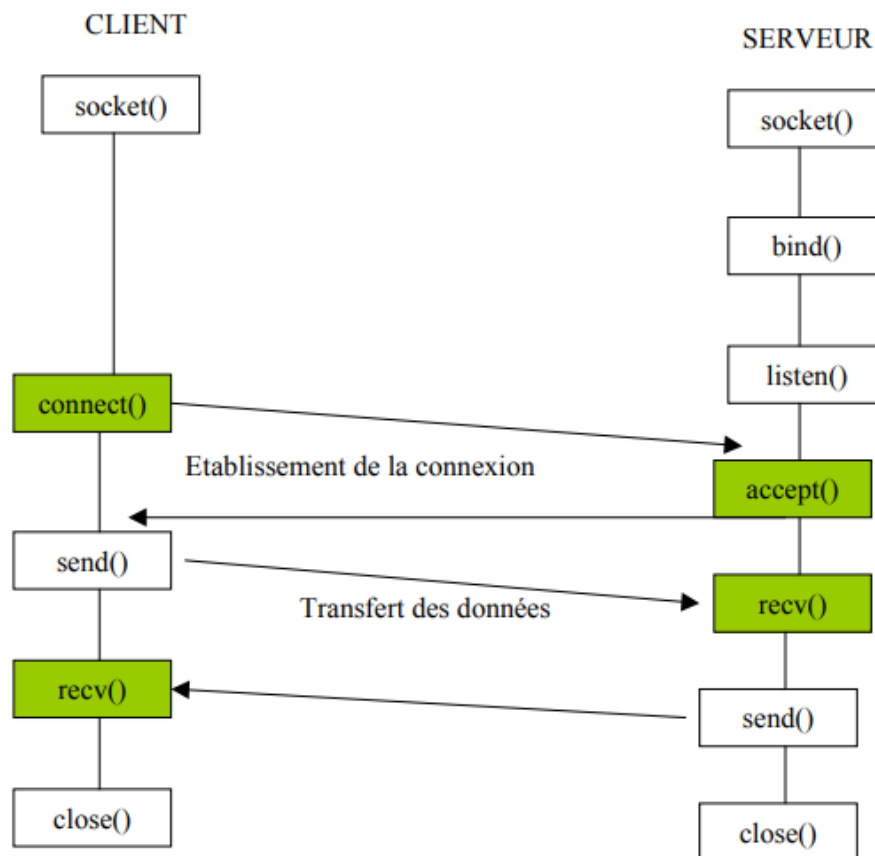


## 2. Les sockets

ooo

---

⇒ Client/Serveur en mode connecté (TCP)

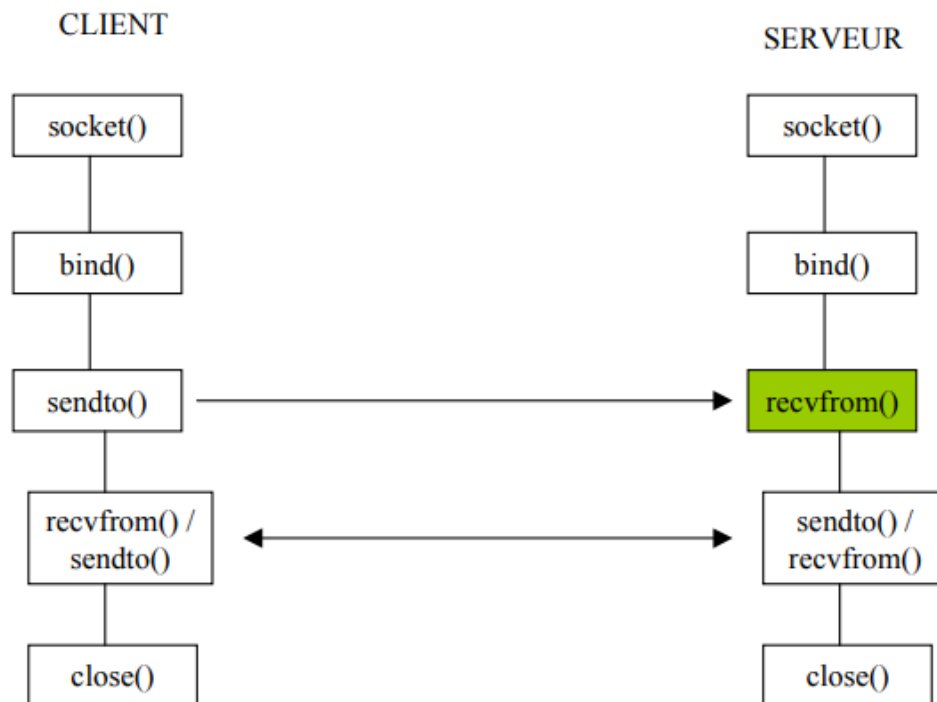


En mode connecté, une connexion doit être établie avant que la transmission effective des données commence, cette connexion est libérée à la fin de la transmission.

## 2. Les sockets

ooo

⇒ Client/Serveur en mode non connecté (UDP)



En mode non connecté, aucune connexion n'est exigée.

A noter que l'implémentation des sockets se diffère d'un langage de programmation à un autre et sous un système d'exploitation à un autre.

## 2. Les sockets

ooo

---

### c. Sockets en C sous Linux

#### ⇒ Création d'une socket

```
int socket (domaine, type, protocole);  
int domaine; /* AF_INET ou AF_UNIX */  
int type; /* SOCK_STREAM (TCP) ou SOCK_DGRAM  
(UDP) */  
int protocole; /* IPPROTO_TCP ou IPPROTO_UDP  
équivalent si = 0 */
```

#### ⇒ Primitive bind (TCP et UDP)

```
int bind (socket, adresse, l_adresse);  
int socket; /* descripteur de la socket  
*/  
struct sockaddr *adresse; /*adresse socket  
(port+adresse) */  
int l_adresse; /* longueur de la structure  
adresse */
```

#### ⇒ Primitive connect (TCP)

```
int connect (socket, adresse, l_adresse);  
int socket; /* descripteur de la socket
```

```
*/  
struct sockaddr *adresse; /*adresse socket  
(port+adresse) */  
int l_adresse; /* longueur de la structure  
adresse */
```

## 2. Les sockets

ooo

---

### ⇒ Primitives listen (TCP)

Mise en écoute : indique que le serveur peut attendre au maximum qlen requêtes des clients.

```
int listen (socket, qlen);  
int socket; /* descripteur socket */  
int qlen; /* nombre maximal de connexions  
traitées */
```

### ⇒ Primitive accept (TCP)

```
int accept(socket, struct sockaddr *pin,  
addrlen));  
int socket; /* descripteur retourné par  
la primitive socket */  
struct sockaddr *adresse; /*initialisé  
à vide et mis à jour par accept avec le  
port et l'ad-  
resse IP de la machine où se trouve le  
client qui a fait un connect */  
int l_adresse; /*longueur de la structure  
adresse*/
```

## ⇒ Primitives envoi & réception en mode TCP

```
ssize_t recv(  
    int descripteur,  
    void *ptr,  
    size_t nb_caracteres,  
    int option /* = 0, MSG_PEEK pour consulter  
    sans extraire, MSG_OOB pour lire une donnée  
    urgente (1 caractère) */  
);  
  
ssize_t send(  
    int descripteur,  
    void *ptr,  
    size_t nb_caracteres,  
    int option /* =0 ou MSG_OOB */  
);
```

## ⇒ Primitive de réception en mode UDP

```
int recvfrom(  
    int descripteur, /* descriptor de la socket  
    */  
    void * message, /* adresse de réception
```

```

*/
int longueur, /* taille zone réservée */
int option, /* 0 ou MSG_PEEK */
struct sockaddr *ptr_adresse, /* adr émetteur */
int *ptr_longueur_adresse /* pointeur sur
long-
ueur zone adresse */
)

```

⇒ Primitive d'envoi en mode UDP

```

int sendto(
int descripteur, /* descripteur de la socket
*/
void * message, /* adresse à envoyer */
int longueur, /* longueur du message */
int option, /* 0 */
struct sockaddr *ptr_adresse, /* adr dest
*/
int longueur_adresse /* longueur adr dest
*/
)

```

⇒ Primitive close (TCP et UDP)

Fermeture de connexion

```
int close (socket)
int socket; /* descripteur de socket */
```

~ o ○ o ~



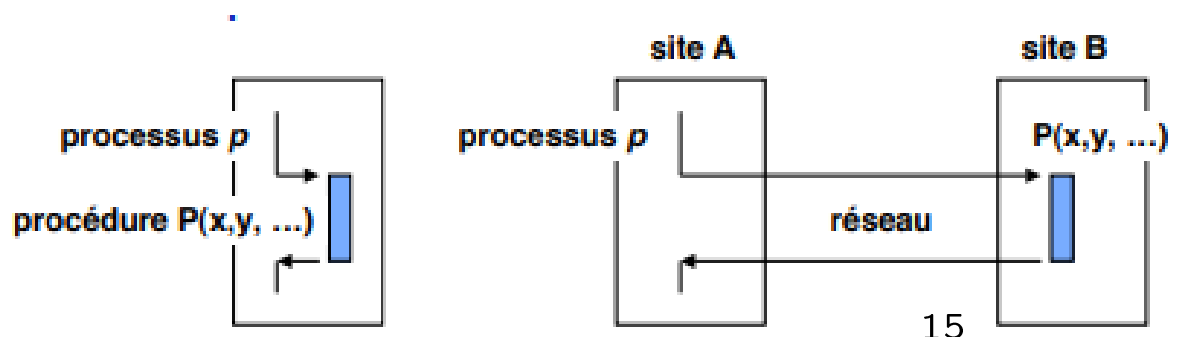
### 3. Remote Procedure Call (RPC)

ooo

---

#### a. Appel de procédure distante

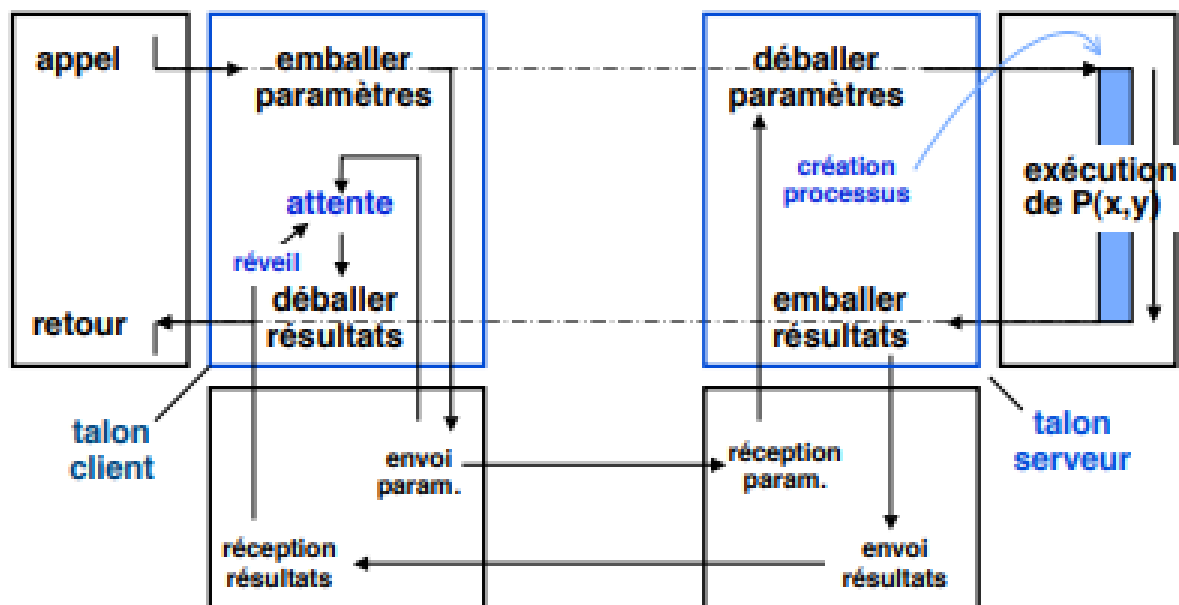
- Appel de procédures locales : **Appelant** et **appelé** dans le même espace virtuel : même mode de pannes, appel et retour fiable.
- Appel de procédures distantes : **Appelant** et **appelé** dans deux espaces virtuels : mode de pannes indépendant, réseau non fiable, temps de réponse.
- **Remote Procedure Call (RPC)** ou appel de procédure distante est une technique de communication inter-processus utilisée pour les applications client-serveur. Il est également appelé *appel de sous-programme* ou *appel de fonction*.



### 3. Remote Procedure Call (RPC)

ooo

#### b. Fonctionnement



⇒ Fonctions des talons (Stub et Skeleton)

- **Talon client (Stub)**

- Représente le serveur sur le site client.
- Reçoit l'appel local.
- Emballe les paramètres.
- Crée un identificateur unique pour l'appel.

- Exécute l'appel distant.
- Met le proc. client en attente.
- Reçoit et déballe les résultats.
- Exécute le retour vers l'appelant (comme retour local de procédure)

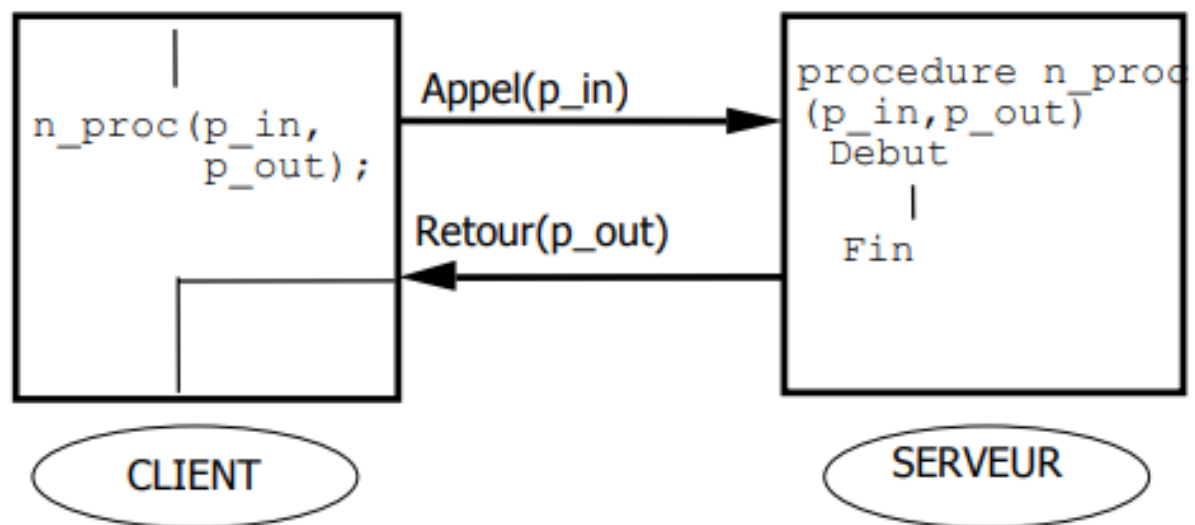
- **Talon serveur (Skeleton)**

- Représente le client sur le site serveur.
- Reçoit l'appel sous forme de message et déballe les paramètres.
- Crée ou sélectionne un processus (ou thread) et lui fait exécuter la procédure.
- Emballe les résultats et les transmet à l'appelant.

⇒ **Mise en œuvre de RPC**

- **Par migration** : Le code et les données de la procédure distante sont amenés sur le site appelant pour y être exécutés par un appel local habituel.

- **Par mémoire partagée** : L'appel distant est réalisé en utilisant une **mémoire virtuelle partagée répartie**. La procédure est installée pour le client comme pour le serveur dans la mémoire partagée répartie.
- **Par messages asynchrones** : Deux messages (au moins) échangés : requête et réponse.



- **Par appel léger (lightweight RPC)** : la communication réseau est réalisée par un segment de mémoire partagée entre le client et le serveur qui contient un

tas pour les paramètres d'appel et de réponse.

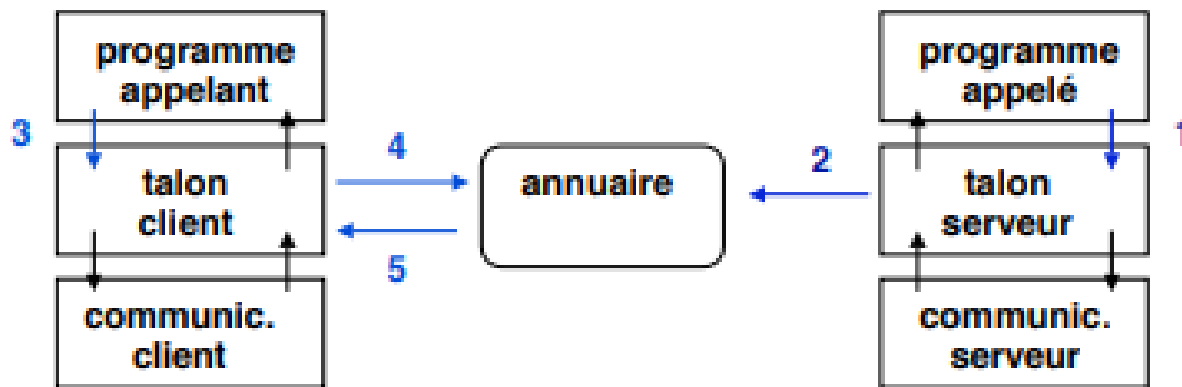
### c. Désignation et liaison

- Comment sont structurés les noms et références permettant de désigner les services distants ?
- Comment localiser une procédure distante ?

### 3. Remote Procedure Call (RPC)

ooo

⇒ Désignation et liaison utilisant un annuaire

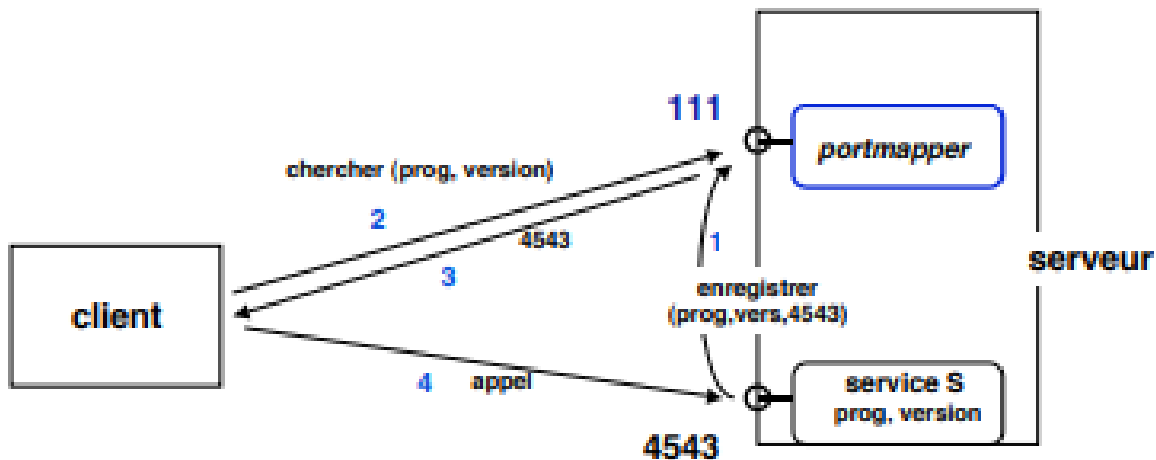


- 1, 2 : le serveur s'enregistre auprès de l'annuaire avec  $\langle \text{nom}, \text{adr. serveur}, \text{n}^\circ \text{ port} \rangle$
- 3, 4, 5 : le client consulte l'annuaire pour trouver  $\langle \text{adr. serveur}, \text{n}^\circ \text{ port} \rangle$  à partir de  $\langle \text{nom} \rangle$

⇒ Désign. et liaison utilisant le portmapper

- Si le serveur est connu (cas fréquent) : on peut utiliser un service de nommage local au serveur, le portmapper.

- Un service enregistre le n° de port de son veilleur auprès du portmapper et le veilleur se met en attente sur ce port.
- Le portmapper a un n° de port fixé par convention (111).



#### d. Développement

Il existe trois façon de développer des programmes utilisant des RPC, utiliser :

⇒ Les fonctions de la **couche intermédiaire**

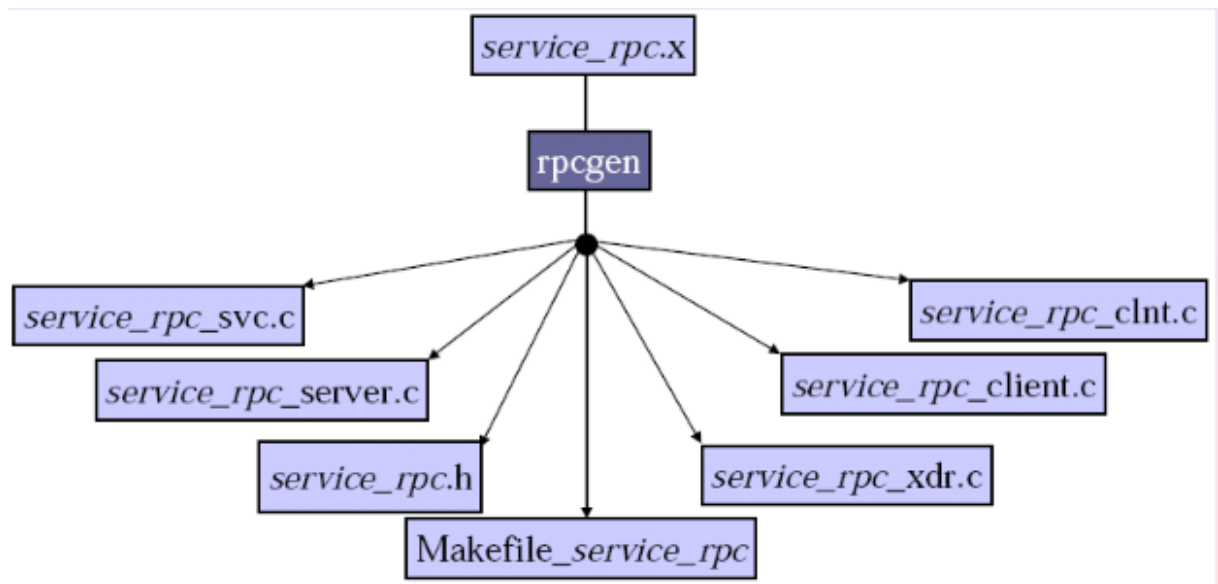
- Elle comporte peu de fonctions.

- Le programmeur est limité dans la configuration du système.
- le programmeur doit développer l'encodage et le décodage.

⇒ Fonctions de la **couche basse**

- C'est un ensemble complet de fonctions.
- Son utilisation est beaucoup plus complexe que la couche intermédiaire.
- À n'utiliser que lorsque le protocole de communication et les délais de temporisation de la couche intermédiaire ne sont pas satisfaisant.

⇒ Le compilateur **rpcgen**





$$\sim \circ \bigcirc \circ \sim$$

## 4. CORBA (Common Object Request Broker Architecture)

ooo

---

### a. Introduction

#### — Problèmes & motivation

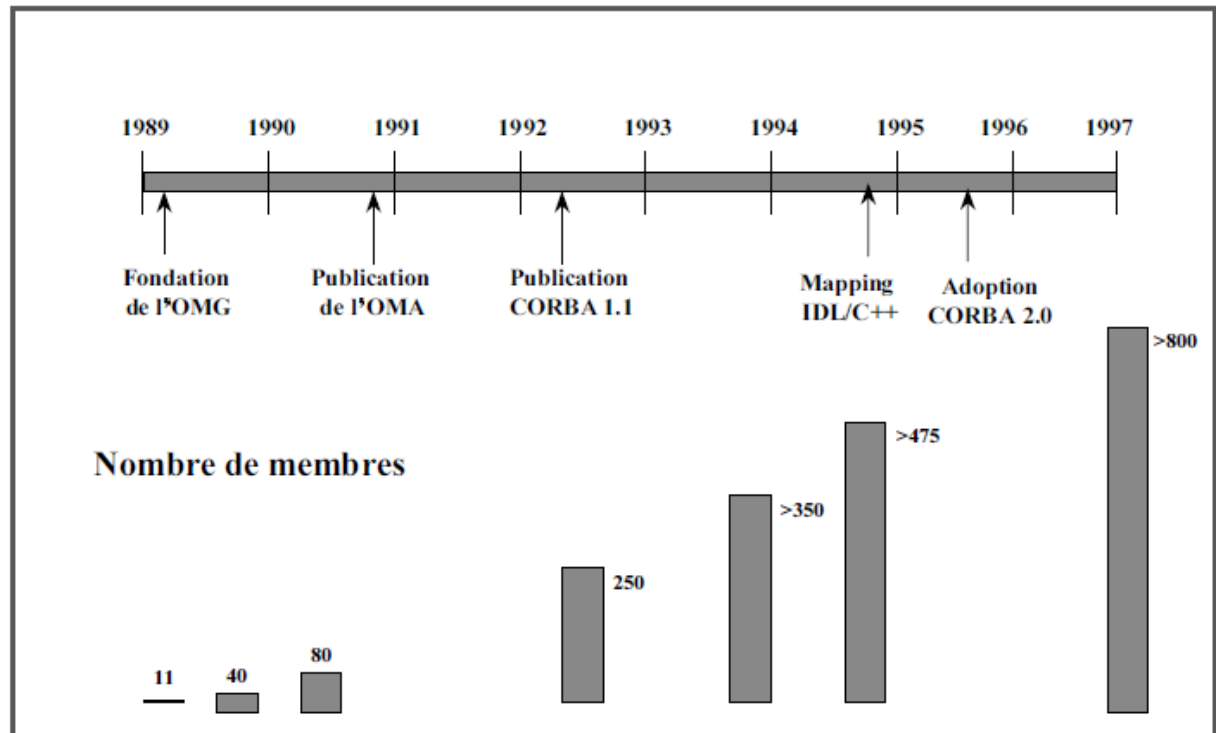
- ⇒ Problème : Intégration des applications, à cause de l'absence d'un consensus pour l'interopérabilité. En effet : Pas de consensus sur les : langages de programmation, plate-formes de développement, systèmes d'exploitation, protocoles réseau, formats des données.
- ⇒ Tirer parti des bonnes propriétés de l'objet (encapsulation, modularité, réutilisation, polymorphisme, etc.).

#### — Objects Management Group (OMG)

- ⇒ Consortium international créé en 1989.
- ⇒ Qui regroupe plus de 800 membres : constructeurs(SUN,HP,DEC,IBM,etc.), environnements systèmes (Microsoft,Novell

outils et langages (Iona, Object Design, Borland, etc.), produits et BD (Lotus, Oracle, Informix, etc.), industriels (Boeing, Alcatel, Thomson, etc.)

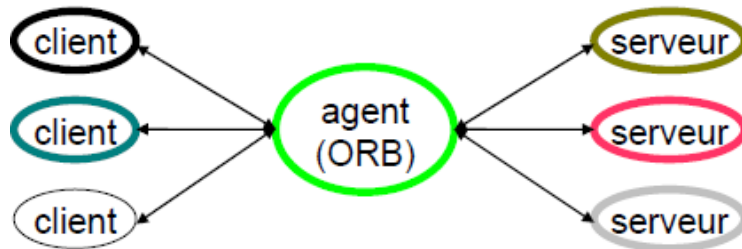
- ⇒ Sa mission est de promouvoir la technologie orientée objet dans les systèmes informatiques distribués.
- ⇒ Fournir une architecture de base pour l'intégration d'applications distribuées tout en garantissant la réutilisabilité, l'interopérabilité et la portabilité. La figure suivante illustre l'historique du OMG.



## b. CORBA : principes généraux

- Met en œuvre le modèle **client-serveur**.
- Introduit une entité intermédiaire : l'agent (**broker**)
  - isole les clients des serveurs.
  - un client peut interagir avec un serveur sans connaître ni son adresse, ni la manière dont il fonctionne.
- **Principe de fonctionnement**
  - le client demande à l'agent l'exécution d'un service (demande d'exécution d'une méthode sur un objet)

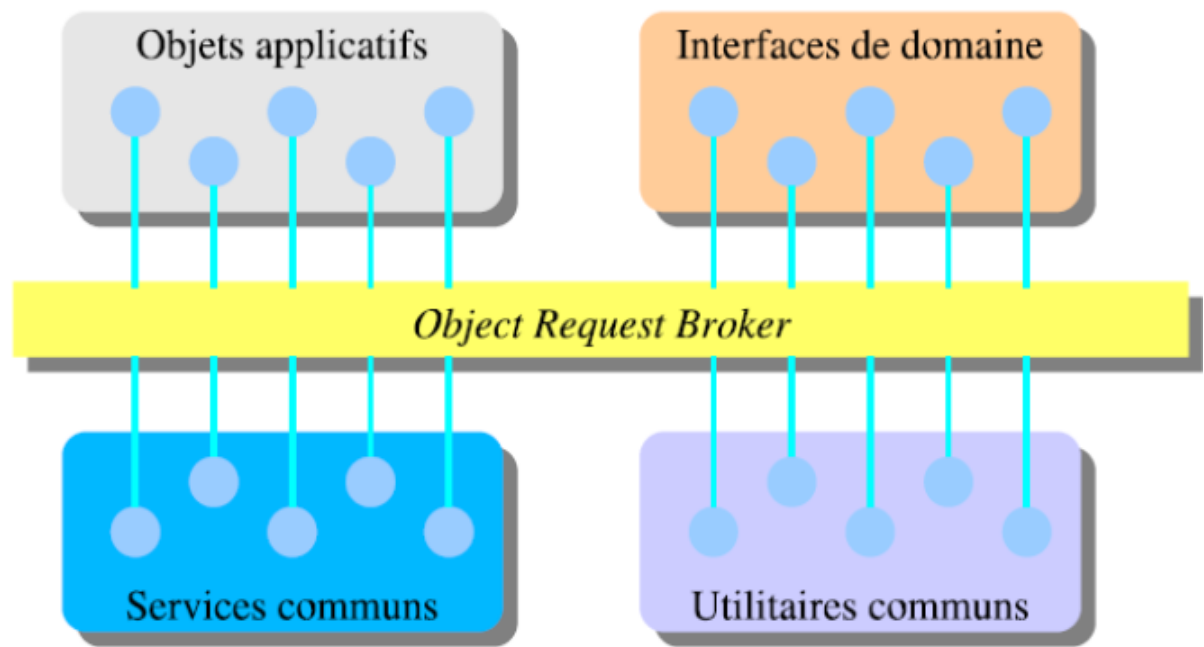
- l'agent identifie un serveur capable de fournir le service.
- l'agent transmet la requête au serveur.



### — Caractéristiques

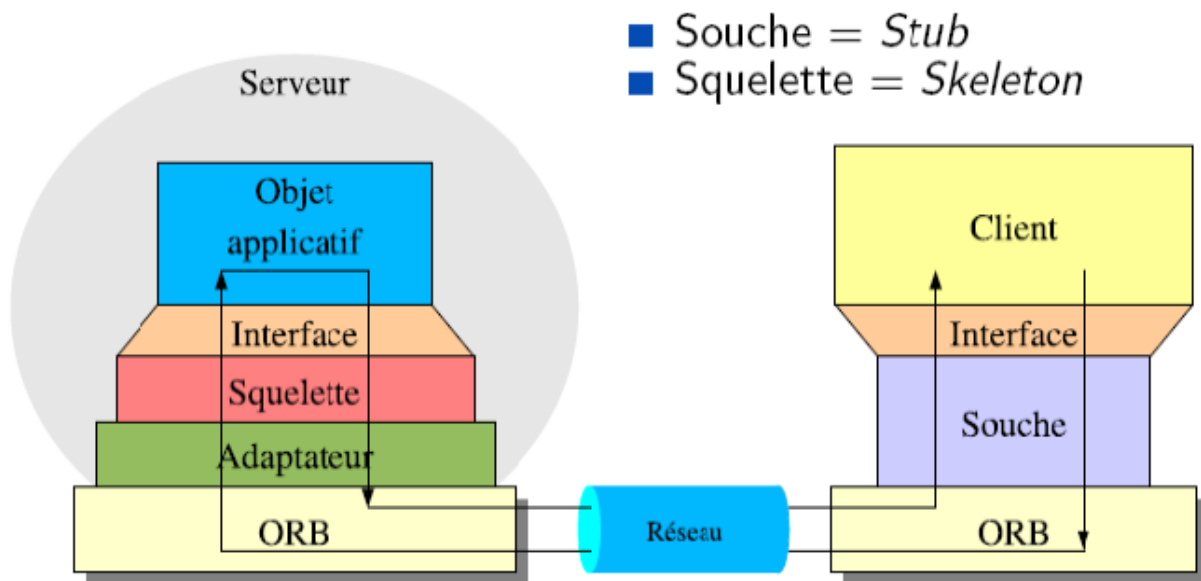
- **Isolation des clients et des serveurs**
  - ils n'ont pas à se connaître mutuellement.
  - permet d'ajouter de nouveaux clients et de nouveaux serveurs sans modifier l'existant.
  - seul l'agent intermédiaire connaît l'adresse et les possibilités de chacun.
- **Communications synchrones ou asynchrones**

- **Client/serveur très dynamique** :  
la même portion de code peut apparaître comme client dans un échange et comme serveur dans un autre échange.
- **Objects Management Architecture (OMA)**
  - ⇒ Il s'agit d'un modèle définissant la façon de décrire des objets distribués dans des environnements hétérogènes.
  - ⇒ Fournir un environnement de développement et d'exécution d'applications réparties portable, cela indépendamment des langages de programmation et des protocoles de communication.



- L'Object Request Broker (ORB ou agent) : Composant central du standard CORBA qui gère : la location d'objet, la désignation des objets, l'empaquetage des paramètres (marshalling), le dépaquetage des paramètres (unmarshalling), l'invocation des méthodes, la gestion des exceptions.
- Les services objets (CorbaServices)
  - services spécifiques à la manipulation d'objets : désignation d'objets, cycle de vie, transactionnel,

- notification d'événements, sécurité, accès multiple, relationnel, gestion des licences d'objets.
- définis dans la Common Object Services Specification (COSS).
- Les services d'application (CorbaFacilities) : ensemble de facilités de haut niveau pour les applications telles que : accès aux bases de données, services d'impression, synchronisation.
  - Les objets d'applications : bibliothèques d'objets permettant d'accélérer le développement logiciel.





## 4. CORBA (Common Object Request Broker Architecture)

ooo

---

### c. OMG Interface Description Language (IDL)

⇒ **Ce langage a été défini :**

- pour décrire les interfaces des objets.
- comme langage pivot entre applications.
- pour générer des squelettes de programme dans les langages de programmation des applications.
- indépendant de tout langage de programmation ou compilateur.
- la correspondance IDL langage de programmation est fournie pour les langages C, C++, Java, Smalltalk, Ada, Cobol.
- **Interface** : ensemble d'opérations et de types  $\iff$  classe C++

*Syntaxe :*

*Interface | [<héritage>] { <interface Body>*

⇒ **Definitions :**

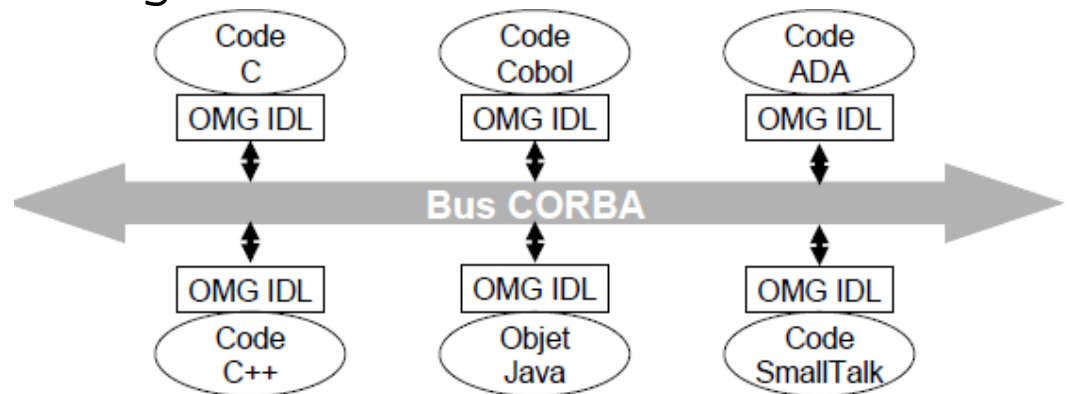
- **Modèle abstrait d'objets** : la façon de décrire des objets distribués.
- **Client** : Entité capable d'émettre des requêtes vers des objets qui fournissent des services.
- **Référence objet** : Objet manipulé par le client pour invoquer des services sur un objet distant : objet implémentation.
- **objet implémentation** : Objet situé sur le serveur qui implémente le code des méthodes des opérations définies en IDL.
- **Requête** : Emise par un client pour demander l'exécution d'une opération sur un objet cible. La requête contient l'opération à exécuter, l'objet cible et les paramètres éventuels.
- **Interface** : Description d'un ensemble d'opérations disponibles sur un objet. Spécification des interfaces en IDL.

- **Opération** : Entité identifiable caractérisée par une signature décrivant les paramètres de la requête et les valeurs de retour.
- **Une signature d'une opération contient**
  - le nom de l'opération.
  - les paramètres (nom formel, type, mode de passage)
  - type du résultat
  - cas d'erreurs ou exceptions
- **ORB : Object Request Broker** : Middleware qui gère les relations client/serv entre les objets.
- **Middleware (Courtier d'objets)** : intermédiaire ou ensemble de logiciels nécessaires pour permettre la communication entre client et serveur.
- **ORB composant central de COBRA qui gère** :
  - la location et la désignation des objets.

- l’empaquetage (marshalling) et le dépaquetage (unmarshalling) des paramètres.
- l’invocation des méthodes et la gestion des exceptions.

⇒ **Une application = des objets CORBA :**

- décrits par des interfaces OMG IDL.
- dialoguant via le bus CORBA.



⇒ **Presentation de IDL : un exemple**

```

#include <date.idl>
module annuaire {
    typedef string Nom;
    typedef sequence<Nom> DesNoms;
    struct Personne {
        Nom nom;
        string telephone;
        ::date::Date date_naissance;
    };
    typedef sequence<Personne> DesPersonnes;

    readonly attribute string libelle;
    exception ExisteDeja { Nom nom; };
    exception Inconnu { Nom nom; };

    interface AdministrationAnnuaire {
        void ajouterPersonne (in Personne personne) raises(ExisteDeja);
        void retirerPersonne (in Nom nom) raises(Inconnu);
    };

    interface ConsultationAnnuaire {
        Personne obtenirPersonne (in Nom nom) raises(Inconnu);
        DesNoms listerNoms ();
    };
};

```

## 5. RMI (Remote Method Invocation)

ooo

---

a. ....

## 6. JAVA EE

ooo



a. ....

## Références

○○○

---